Warsaw University of Technology
Faculty of Mathematics
and Information Science

Master's diploma thesis

in the field of study Computer Science and Information Systems
and specialisation Artificial Intelligence

Multi-model LLM system with dynamic planning and model selection optimization

Jakub Knyspel

student record book number 313282

thesis supervisor
dr hab. Maria Ganzha, prof. nzw.

WARSAW 2026

Abstract

Multi-model LLM system with dynamic planning and model selection optimization

TODO

Keywords: TODO, ...

Streszczenie

Implementacja wielomodelowego systemu LLM z dynamicznym planowaniem i optymalizacją doboru modeli

TODO

Słowa kluczowe: TODO, ...

Contents

1. Introduction **11**

2. Related Work **12**

2.1. RouteLLM 12

2.2. RoRF - Routing over Random Forests 12

3. System Requirements & Constraints **13**

4. Description of the System **14**

4.1. API Interface 14

4.1.1. Security 14

4.2. LLM integration 15

4.3. Task Execution Process 15

4.3.1. Planning and Reevaluation 16

4.4. File Handling 18

4.5. Asynchronous Processing 19

4.6. Error Recovery 20

4.7. LLM Selection 20

4.7.1. Cost Estimation 22

4.7.2. Reasoning Tokens 23

4.7.3. Selection Restrictions 24

5. Scoring **25**

5.1. Dataset Selection 25

5.2. Preprocessing & Feature Selection 26

5.2.1. Complexity Features 26

5.2.2. Domain Scores 27

5.2.3. Style Features 28

5.2.4. Task Type Features 28

5.3. Data Analysis & Transformations 29

5.4. Prompt-invariant Scoring . . . 29
5.4.1. Iterative Models . . . 30
5.4.2. Non-iterative Models . . . 30
5.5. LLM Embedding . . . 31
5.5.1. Metrics . . . 32
5.5.2. Features . . . 32
5.5.3. DNN-based Embedding . . . 33
5.5.4. Tuning-based Embedding . . . 33
5.5.5. Attention Embedding . . . 33
5.5.6. Prompt-invariant Embedding-based Models . . . 34
5.6. Simple Models . . . 35
5.6.1. Gradient Boosting Model . . . 35
5.6.2. DNN Model . . . 37
5.7. Complex Models . . . 41
5.7.1. Transformer Model . . . 41
5.7.2. Response Predictive Model . . . 45
5.7.3. Multistage Models . . . 49
6. Response Length Prediction . . . 51
6.1. Input and Output Data . . . 51
6.2. Models . . . 51
6.2.1. Gradient Boosting Model . . . 51
6.2.2. DNN Model . . . 51
7. Analysis of the Solution . . . 52
8. Conclusion . . . 53

1. Introduction

TODO

2. Related Work

2.1. RouteLLM

TODO

2.2. RoRF - Routing over Random Forests

TODO

TODO: More references

3. System Requirements & Constraints

TODO: What did I decide to do? What constraints did I settle on?

4. Description of the System

The system was implemented as a REST API application, which allows users to create a task, monitor it's state, and receive the result. [TODO: update?] For simplicity, no UI was implemented - the system can be controlled by manual HTTP requests instead. While implementing a user-friendly interface would be an improvement, it isn't too interesting in the context of the problem, while relative simplicity of implemented operations makes it not required for testing and demonstration purposes.

4.1. API Interface

In it's essence, the task completion process is very simple from user perspective. The only information that needs to be provided by the user is the task prompt, and files required for task completion. These actions were implemented as two `POST` endpoints. Supplementary `GET` endpoints on the same routes make it possible to inspect the state of running and completed tasks, completing the minimal API design required for task execution.

In practice, a few additional endpoints are required. The following routes were implemented:

- `/tasks`: Creating new task, inspecting existing tasks;
- `/files`: Uploading a file with metadata, inspecting uploaded files;
- `/sse/events`: SSE event stream (described in section 4.5);
- `/allowed-models`: Setting a list of allowed LLMs for step completion (described in section 4.7.3).

Presented list does not describe all endpoints that were implemented. Routes not directly related to task completion, related to API key management etc., were omitted for brevity.

4.1.1. Security

The system uses API key authentication. Every request requires the key to be present in `Authentication` HTTP header, validating access rights, and allowing for attribution of the

request to the user whose API key was used. The API exposes a set of API key management endpoints under `/api-keys` route, allowing for listing names of existing API keys, creating new keys, and deleting them.

4.2. LLM integration

[TODO: add some reference] Integration with LLM providers is a central part of the project. Contemporary commercial LLM landscape features a high number of providers, each offering a selection of models. While certain conventions exist (for example, `openai` integration packages for various programming languages are widely adopted by different providers, offering compatible API interface), they cover only a part of functionality required by the system. While chat and one-shot completion APIs are standardized, aspects like per-request configuration or pricing information are not.

This problem is solved by *OpenRouter*. *OpenRouter* is an LLM gateway, offering unified API access (compatible with `openai` interface), request configuration, LLM description listing, and pricing information. This effectively means that instead of requiring a separate integration for each supported LLM provider, only a single implementation is required, drastically reducing complexity.

Usage of *OpenRouter* also simplifies API key management. In case of direct integrations, a separate API key would be required for each supported provider. Instead, a single *OpenRouter* API key can be used for all LLM requests. This simplification makes it possible to skip implementation of API key management services in the system altogether, instead requiring the API key to be included in API requests that require LLM usage, never saving it to a persistent location server-side.

4.3. Task Execution Process

Diagram 4.1 provides a high-level overview. Task completion process starts with user *POST* request to `/tasks`, specifying the prompt, and a list of uploaded files that are required to successfully complete the task. Subsequently, step list is created based on the prompt. Steps are completed in order, based on dependencies between them. For each, a different LLM is selected to complete it, based on the prompt and estimated request cost (described in section 4.7). Some steps may define a planned re-evaluation point, in which case remaining step list is updated,

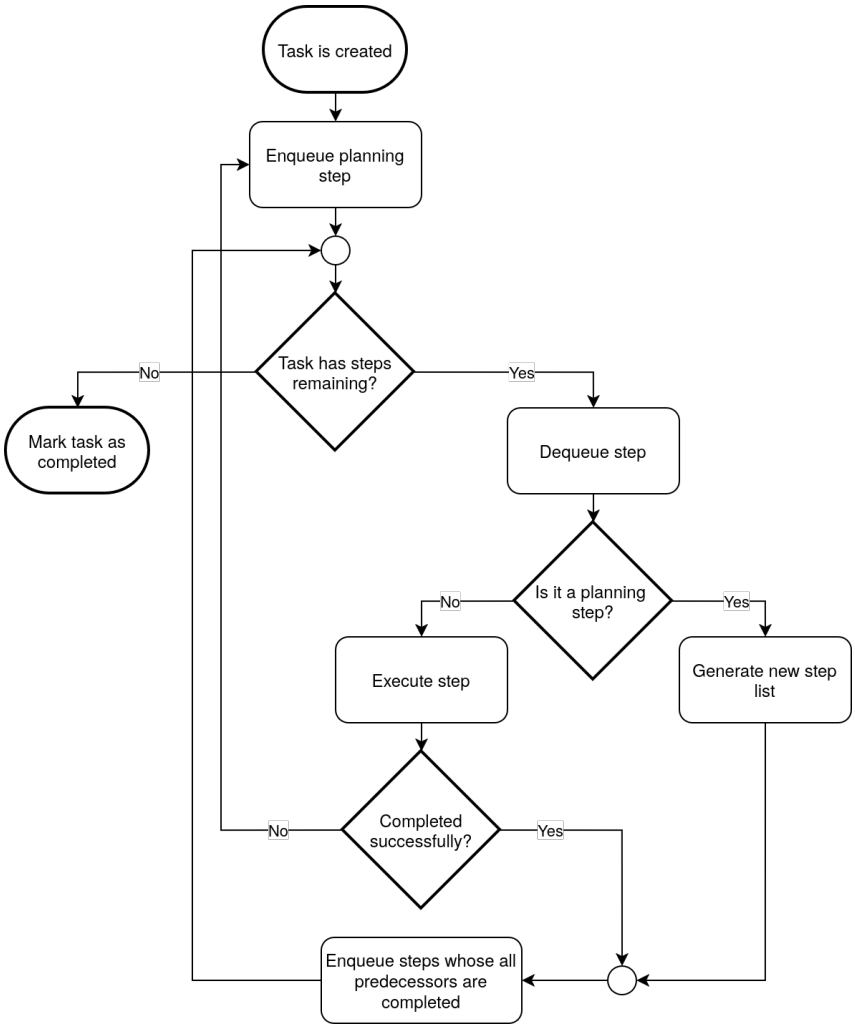

Figure 4.1: Task execution flow

based on the result of already completed steps. The same happens if step completion fails (section 4.6). Finally, the last step produces a text output, which is returned as the result of the task.

4.3.1. Planning and Reevaluation

TODO: some references etc.

Producing a set of steps required for completion of a given task can be very easily framed as a text processing task requiring a certain level of intelligence and reasoning. Therefore, the simplest solution to that problem is just using a sufficiently good, commercial LLM. While using a high-end LLM model may seem contradictory to the objective of reducing task completion cost, the decision is justified by the fact that step planning prompt is relatively short (compared to step prompts, which may contain much more information), and that the request doesn't contain any input images or files, which cause a dominant contribution to request cost. Planning shouldn't

require too extensive reasoning, either, as the actual complex task completion work should be performed when steps are executed.

In initial planning request, the prompt contains task description specified by the user, along with a list of metadata of files attached to the task - their names, types, and user-specified descriptions. In case of reevaluation, outputs of already completed steps are also specified, along with failure reason of the task whose execution failed, if reevaluation was triggered by step failure. Requested output format is a JSON list of step definitions. Each step definition specifies a prompt, previous steps that it depends on (and whose outputs will be included when that step is processed), required files, and required LLM capabilities.

Requested output specifies a number of constraints that LLM response must satisfy, both defined by the format (JSON schema), and by semantic rules (step dependency graph must be acyclic, etc.). These constraints are validated when response is received. If the format is not correct, task status is set to failed and processing stops. While a retry mechanism could be easily implemented, it turned out to be unnecessary - modern commercial LLMs seem to be sufficiently capable of producing a structured output that follows such rules, making failures during planning step extremely rare.

LLM Capabilities

Different LLMs support different mechanisms and options - some allow image input, others implement native web search, etc. Those "hard" capabilities of models may be crucial for completion of certain steps, so they can be specified in step definition by planning model. Often, those capabilities serve as filters - if a step requires certain mechanisms or input modalities, LLMs that don't implement them cannot be used to complete that step. That's not always the case, though. *OpenRouter* implements a range of simple options available regardless of requested model, which can serve as replacements for mechanisms that are not supported by a given LLM. These include PDF processing options, prompt-based web search, etc. If such capabilities are required by step definition, they don't cause any models to be filtered out, but instead cause a specific option to be included in the request, specifying usage of a certain mechanism.

The following capabilities are implemented:

- **Reasoning**: requires an LLM that supports reasoning to be used, and requests medium-effort reasoning during step completion;
- **EXA web search**: requests usage of prompt-based EXA web search during step completion;

- **Native web search**: requests usage of LLMs that support native web search (i.e. *Gemini* and *Perplexity* models);
- **OCR PDF processing**: specifies that attached PDF files should be processed via OCR, using an *OpenRouter*-managed mechanism;
- **Text-based PDF processing**: specifies that attached PDF files should be processed by simply extracting included text;
- **Native PDF processing**: requires usage of models that accept PDF as native model input;
- **Input modalities**: requires usage of models whose input modalities cover all types of attached files (implicitly specified by a list of files required for step completion).

4.4. File Handling

Files are uploaded independently from tasks. During upload, in addition to file contents and standard file upload metadata, request specifies declared content MIME type, optional description, and optional user-specified metadata.

The upload is followed by type-specific preprocessing. This step performs required type conversions, text extraction, and metadata computation. Text extraction is performed for supported Office XML formats (`.xlsx`, `.docx`) - as most LLMs don't support these formats as input, such files are converted to text using `docx` and `openpyxl` packages.

As for metadata computation, the most important computed value is token count of text data. Depending on the format, text-based files are either priced as input tokens (i.e. PDFs if text-based processing is selected), or simply included in the prompt (i.e. plain-text or markdown files). This means that knowing the token count of their contents is required for accurate request cost prediction (described in detail in section 4.7.1).

Computed metadata are appended to user-specified additional metadata. Finally, the file is saved to disk, (encoded as Base64 for security), and metadata is stored in the database.

In addition to actual files, system supports registering file URLs. LLMs generally allow to specify URLs as input in case of images, video, audio, and PDF files, so supporting this format in the system was a natural consequence. URL registration process is analogous to file upload, with exception of metadata computation. In this case, as file contents aren't available, data required for cost estimation is expected to be provided by the user in additional metadata.

The following file types are supported:

- Image: `PNG`, `JPEG`, `GIF`, `WebP`;
- Video: `MP4`, `MOV`, `MPEG`, `WebM`;
- Audio: `MP3`, `MPEG`, `WAV`;
- PDF files;
- Office formats: `XLSX`, `DOCX`;
- JSON files;
- All text-based formats: MIME types starting with `text/` prefix;

4.5. Asynchronous Processing

Task processing, as pretty much any operation that uses LLMs, is naturally an asynchronous process. LLM requests take several seconds to complete, which means that complex multi-step tasks may take multiple minutes to finish. Design of the system reflects such nature of the problem.

`POST /tasks` endpoint implements a common convention for long-running operations, returning `202 Accepted` immediately, rather than waiting for task to complete before returning a response.

Step completion is implemented using a work item queue, with state saved in the database. When new task is created, initial planning step is added to the queue. Subsequently, when any step is completed, all task steps for which all predecessors were completed are added to the queue, ensuring that task execution continues. This design ensures that the execution is resumable when system is restarted, while also making process thread count independent from the number of running tasks.

SSE

HTTP communication is asymmetric by design - it always starts with a request from the client, which the server handles, returning a response. This property makes dealing with long-running asynchronous processes inconvenient - the only way for the client to wait for an operation to complete is to use a pooling loop, checking the state every so often. This problem can be quite easily solved by implementing a non-HTTP notification mechanism (i.e. WebSockets [TODO: ref]). However, a simpler solution exists: Server-Sent Events [TODO: ref]. SSE stream utilizes

HTTP for server-sent notifications, by simply keeping the HTTP connection open, writing each event data as a JSON line. While not aligning with HTTP design, this mechanism provides a simple alternative to pooling or using additional connection protocols.

The system exposes an SSE endpoint at `/sse/events`. Events are produced when task state is updated in any way (step is completed, step list is generated, task finishes, etc.). Additionally, metadata filtering is implemented, making it possible to request only a certain subset of emitted events - for example, only those related to a particular task.

4.6. Error Recovery

LLM-based operations require a relatively extensible error handling. First, regardless of format schemas and specifications, LLM output should be treated as text, which may or may not satisfy requested constraints. Failed format validation during step execution causes the step to be failed, triggering a reevaluation step.

Second, LLM may simply be unable to complete a step. Steps, just like tasks, aren't constrained in any way, and may specify problems that are extremely hard to solve, or which don't make sense after acquiring certain information from previous steps output or file contents. A plan for a certain mathematical task may contain a step requiring a proof of Riemann Hypothesis. Image recognition task may specify a step requiring a model to determine a scientific name of the highest tree on an attached photo, while the photo does not contain any trees. To solve this problem, step output schema specifies a "failure reason" field, and LLM that executes the step is instructed to use it if the step cannot be completed to specify the problem. Failed step subsequently causes reevaluation to be performed. Reevaluation prompt contains the failure reason, making it possible to plan subsequent steps in a way that solves the problem.

4.7. LLM Selection

Each step requires different LLM capabilities and input modalities to be completed. In addition to these constraints, each viable LLM has different strengths and weaknesses, as well as a different request cost. The goal of LLM selection for a given step is choosing an LLM which satisfies all input modality and capability constraints, while maximizing a certain utility function, combining cost minimization requirement with maximizing prompt-specific expected response quality score.

Ensuring that chosen LLM satisfies input modality and capability constraints is done by simply filtering the list of available models based on their metadata. If no model satisfies these constraints, step is failed with a descriptive reason, and reevaluation is triggered.

The choice of utility function defines the behavior of the system. For example, higher emphasis on predicted quality score will cause the system to favor LLMs which receive higher scores, regardless of estimated cost. Despite it's importance, there is no obvious criterion that can be used to determine it's quality. Fundamentally, quantities that the utility function balances aren't comparable. While it would be possible to construct some arbitrary metric and find an utility function that is the best according to that metric, there's no good reason to consider such process to be better than specifying chosen utility function directly - in principle, for any utility function it would be possible to choose such a fitness metric that chosen utility function fits the constraint the best.

Due to aforementioned reason, utility function was chosen explicitly. In practice, while the choice is important, it does not impact the implementation and design of other parts of the system - if necessary, changing utility function is extremely simple.

Chosen metric is described by equation 4.1. S and C represent predicted quality score and estimated request cost, normalized to mean 0 and standard deviation equal to 1 across all LLMs that satisfy capability and input modality constraints. λ, k_c, k_s, τ_c, τ_s are parameters.

$$U(C, S) = S - C - \lambda \operatorname{softplus}\big(k_c(C - \tau_c)\big) - \lambda \operatorname{softplus}\big(k_s(-S - \tau_s)\big) \tag{4.1}$$

The main utility criterion is defined by $S - C$ term - relative score difference from the mean is treated equally to relative cost difference. Subsequent softplus terms discourage the choice of LLMs with extremely bad score regardless of the cost, and extremely expensive LLMs regardless of the score. λ defines the strength of penalty terms, τ_c and τ_s - extreme value cutoffs (in standard deviations, since S and C are normalized), k_c and k_s - steepness of penalty values past the cutoff. [TODO: update utility function in code]

The following parameter values were chosen:

- $\lambda = 5$,
- $k_c = k_s = 3$,
- $\tau_c = \tau_s = 2$

Figure 4.2 shows values of chosen utility function for normalized score and cost values between -4 and 4. Introduction of softplus penalty terms produces steep negative slopes where cost or score values are too unfavorable.

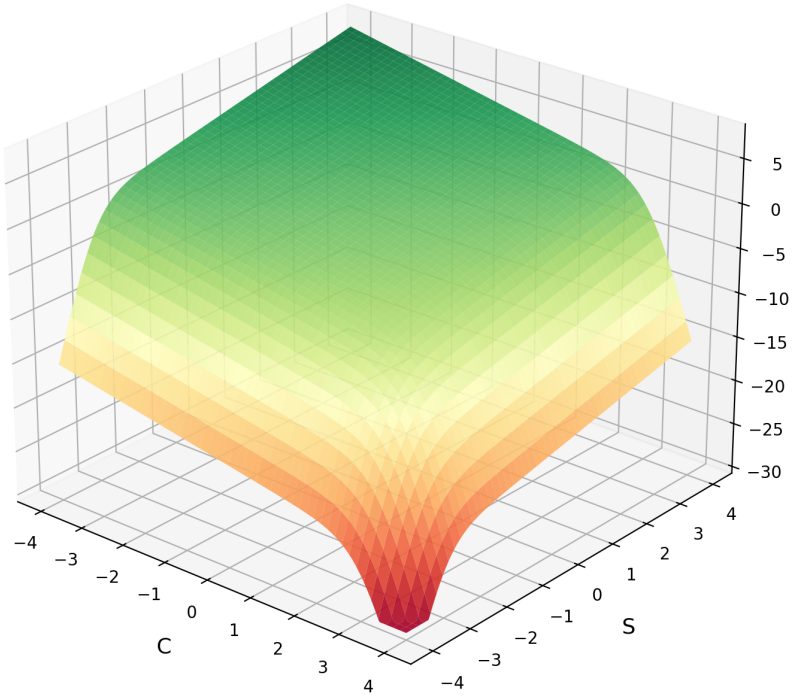

Figure 4.2: Utility function values

4.7.1. Cost Estimation

Estimating the cost of LLM request is relatively simple due to well documented pricing rules exposed by *OpenRouter* API, save for a few problematic aspects. In implemented system, request cost consists of the following components:

- Input tokens,
- Output tokens,
- Reasoning tokens,
- Web search,
- Files.

Input Tokens

Cost of input tokens can be estimated to quite high accuracy, because the text of the prompt is known before request is made. As tokenization is a cheap operation in terms of compute time, it can be performed on the prompt by the system to find token count. Computed value

depends on tokenizer that is used, and as commercial LLM tokenizers aren't publicly available for all models, it's not possible to know input token count exactly. However, the quantity can be estimated simply by using a different, accessible tokenizer implementation. The system estimates token count using `tiktoken` implementation, used by OpenAI LLMs.

Output Tokens

Output token count is unknown before the request is made. As output token cost contribution may be dominant in the final request cost, a machine learning prediction mechanism was implemented, which attempts to guess output length based on chosen model and input prompt. It is described in section 6.

4.7.2. Reasoning Tokens

Similarly to output token count, reasoning token count is also unknown before request is made. In this case, however, there are no datasets that can be used as training data for a potential machine learning solution. Because of that, the system estimates reasoning token count as a proportion of predicted output token count, with multiplier depending on requested reasoning effort. An argument can also be made that reasoning token cost will likely influence the total cost to lesser degree due to significantly cheaper per-token cost relatively to output tokens.

Web search

The system specifies two modes of web search: EXA prompt-based search, and native provider-side web search. Cost estimation is easy in case of EXA search. According to *OpenRouter* documentation, EXA search is run before LLM request is made, retrieving a specified number of search results based solely on the LLM prompt, that can serve as additional context. Since the number of results is configurable, and price per result is known, EXA search cost can be computed exactly.

In case of native web search, while the price per search tool invocation is known, the number of web search invocations cannot be configured in general. To estimate the cost, an instruction not to use more than a specified number of web search calls is added to LLM prompt. Subsequently, web search cost can be estimated by assuming that the specified number of search invocations will be performed.

Files

Cost of file inputs depends on their type.

- **Images:** All models specify a set price per input image, making exact cost computation trivial.
- **Text-based files:** Text files, and files converted to text during preprocessing (like Office XML formats), are included in input prompt, meaning that they influence the cost by incrementing the number of input tokens. Their cost can be estimated in the same way as input token cost - by tokenization.
- **Audio:** Audio input is generally priced as input tokens, at the same per-token rate. [TODO: maybe just tokenize?]
- **Video:** Video input pricing is not returned by *OpenRouter*. Lack of any pricing information severely limits possible estimation methods. Due to these factors, only a rough estimation based on file size and image input price could be implemented.
- **PDFs:** PDF input files are priced differently, based on requested processing mode. If text processing is chosen, text content is extracted from the file and priced as input tokens. Total text token count is estimated during file preprocessing, in the same way as prompt token count. If OCR processing is requested, PDF is priced per page - page count is computed together with text token count during preprocessing. In case of native processing, files are tokenized in model-specific way, making exact computation impossible. File token count is estimated based on text content and number of images contained in the PDF.

4.7.3. Selection Restrictions

[TODO: reference assumptions/introduction] The system is designed to be independent from the set of LLMs whose prompt and response data was included in the dataset used for scoring model creation, handling unknown LLM IDs and assigning scores based on certain mean LLM representation. This requirement ensures that the set of LLM models available through *OpenRouter* doesn't need to be restricted in any way. While useful in practice, this characteristic is undesired when testing the system. For that purpose, a possibility of limiting the set of available LLMs was introduced. If the list of allowed model names is set through an API endpoint, Only specified LLMs will be used for step completion. This ensures that, when the system is tested, all available LLMs can be scored without falling back to average LLM representation.

5. Scoring

TODO

5.1. Dataset Selection

The title of the most popular human-evaluated LLM preference dataset without a doubt belongs to *Chatbot Arena Conversations* [TODO: ref]. It is used as primary human-evaluated dataset in referenced works [TODO: verify and ref].

Chatbot Arena Conversations data consists of a pair of responses from different LLM models to a single user prompt, along with a comparison label, determining which model's answer is considered better by the user. When comparison is unclear, the label can also indicate that both responses were sufficient for a given query, or that neither model managed to produce a good answer. Selection of LLMs is sufficiently wide for a model to learn a broad spectrum of differences between different LLM models, while prompts span a variety of topics.

However, there is a crucial issue: published in 2022 [TODO: verify], *Chatbot Arena Conversations* dataset only includes comparisons between rather old LLM models. This isn't a disqualifying problem, as my work doesn't focus on particular LLM models. Though, A decision to use this dataset would require two important implications: that human preference during evaluation of a pair of responses remains unchanged, and that the general distribution of responses produced by LLMs is similar.

It's hard to evaluate how close to the truth the first assumption is. While human preferences likely did not change drastically over the last four years, expectations of what LLMs should be able to do did undergo an undeniable change, simply due to their capabilities increasing. Another significant difference may be caused by a wider adoption - people whose responses were used in *Chatbot Arena Conversations* dataset belonged to a significantly narrower group of "early adopters" of LLM technology, compared to a group of LLM users today.

These reasons prompted me to decide to rely on a newer alternative to *Chatbot Arena Conversations* dataset. Collected and released by the same organization (*LMSYS* [TODO: ref]),

LMArena Human Preference is a successor of aforementioned dataset. It's structure closely resembled *Chatbot Arena Conversations*, also including human comparison results over a pair of LLM responses to the same prompt. Additionally, the dataset includes a range of common prompt labels [TODO: describe], which can be used as additional features in any prediction task. *LMArena Human Preference* is also an improvement in terms of size. It contains around 140 thousand comparison pairs (compared to *Chatbot Arena Conversations*'s 33 thousand), and includes responses from 53 different models (compared to *Chatbot Arena Conversations*'s 20).

It's important to note that due to a very similar structure, these datasets can be used interchangeably. Due to previously mentioned reasons, mainly *LMArena Human Preference* was used for model training and evaluation throughout this work. However, comparisons between datasets are made where relevant, highlighting important differences between their data.

5.2. Preprocessing & Feature Selection

Intuitively, when it comes to inferring the quality of LLM's response to a given input, the most important aspect of that prompt should be it's semantic meaning - ideally, prompts that are equivalent when it comes to meaning, differing only in wording, should produce the same answer. Semantic meaning of a given text can be captured via a range of specialized text embedding models, which, given an input, return a vector that should be useful for a range of classification or regression tasks. [TODO: references, examples]. As such, prompt embedding vectors are a primary candidate for model input features.

In practice, assumption that prompt features which are unrelated to it's meaning shouldn't influence LLM output isn't reasonable. LLMs in they current state are far from ideal, so certain biases and simple relationships cannot be ruled out. Thus, instead of relying solely on prompt embedding vectors, input features also contain a set of supporting values that represent certain characteristics of submitted question.

Aside from prompt embedding vectors, the following simple features were used:

5.2.1. Complexity Features

8 features encoding prompt complexity were defined:

- **Character count**;
- **Word count**;
- **Sentence count**;

- **Average word length**;
- **Question density** - Number of question marks in the prompt, divided by prompt length;
- **Nesting density** - Number of parentheses, divided by prompt length;
- **Multi-part indicator density** - Number of words indicating multiple aspects/parts in the prompt ("and", "also", etc.), divided by prompt length;
- **Inverse vocabulary richness ratio** - $1 - \frac{[\text{number of unique words}]}{[\text{word count}]}$.

Features that don't directly encode prompt length were normalized by that length, in order to reduce dependency between them - for example, it's reasonable to assume that longer prompts will, on average, contain more parentheses and question marks than shorter prompts.

Vocabulary richness ratio was inverted by subtracting it's value from 1. This transformation was performed to make value distribution more similar to other features, which made it possible to unify it's further processing with other features [TODO: ref]. In an overwhelming majority of samples, nearly all words are different, causing vocabulary richness ratio to primarily take a value of 1 (or close to it). "Inverting" values causes them to instead be 0 (or close to it).

5.2.2. Domain Scores

12 features describing prompt domain were defined:

- **Science**;
- **Medicine**;
- **Law**;
- **Finance**;
- **Tech**;
- **Academic**;
- **Casual**;
- **Formal**;
- **Philosophical**;
- **Historical**;
- **Personal**;

- **Business**;

Each domain score was defined as a ratio of words related to that domain, to total word count of the prompt.

5.2.3. Style Features

6 style features were defined:

- **Informal marker density** - Number of informal style indicators (like "btw", "tbh", etc.), divided by prompt length;
- **Specificity marker density** - Number of words indicating constraints and requirements (like "exactly", "must", etc.), divided by prompt length;
- **Politeness marker density** - Number of words indicating polite tone (like "please"), divided by prompt length;
- **Urgency marker density** - Number of words indicating urgency (like "quickly", "fast", etc.), divided by prompt length;
- **Starts with verb** - Boolean feature indicating that first word of the prompt is a verb;
- **Is a question** - Boolean feature indicating that the prompt ends with a question mark;

5.2.4. Task Type Features

10 boolean features defining type of the task given by a prompt were defined:

- **Expects code** - Whether prompt contains words indicating that code should be returned;
- **Contains code** - Whether prompt contains code blocks;
- **Expects math** - Whether prompt contains words indicating that some mathematical calculation should be performed;
- **Contains numbers** - Whether prompt contains numbers;
- **Contains math symbols** - Whether prompt contains common math symbols (i.e. equality symbol);
- TODO: Continue

TODO: List remaining features

5.3. Data Analysis & Transformations

Feature transformations were defined based on their value distributions. Two kinds of transformations were defined for numeric data, described further in this section. Aside from these transformations, all numeric features were scaled to mean 0 and standard deviation equal to 1.

First, certain features show exponential behavior. Features related to prompt length are a perfect example of that [TODO: ref some plot]. Those features were transformed using a logarithm [TODO: reword] to achieve a more uniform value distribution. Small values are clipped to 10^{-6} to prevent them from exploding to $-\infty$.

Second, certain numeric features are dominated by zero values. Domain scores are a good example. Most prompts don't refer to anything related to finance. While distinguishing between prompts where financial language dominates from prompts that contain a handful of financial terms might be significant, simply transforming the data via a logarithm causes prompts without financial language to have scores which are similar to prompts containing only a few phrases related to finance, making those two cases hard to distinguish. Additionally, zero value dominance will negatively impact scaling. To address these issues, the transformation described by equation [TODO: ref] was introduced. Additionally, to address the scaling issue, zero values were excluded from scaling - mean and standard deviation necessary for scaling was computed from remaining values.

TODO: equation

Transformation is applied after scaling. Softplus operation transforms non-zero data from $(-\infty, \infty)$ range onto $(0, \infty)$, while keeping the monotonicity [TODO: beter word] relatively unchanged. To ensure a clear separation between zero and small but nonzero samples, zero values are encoded as -1.

[TODO: plots for each feature + chosen transforms]

5.4. Prompt-invariant Scoring

A natural simplification of prompt routing problem comes from an assumption that LLM performance does not depend on the prompt. In this case, every LLM has a single score associated with them, and the problem reduces to assigning those scores based on comparison data. While this simplification makes the problem much less interesting and complex, it is more commonly used when ranking LLMs, due to capturing a general LLM quality rather than focusing on particular cases [TODO: examples, refs]. Also, exploration of this problem will serve as a good

baseline for prompt-aware scoring.

5.4.1. Iterative Models

TODO (do I need this section?)

5.4.2. Non-iterative Models

The problem of scoring entries based on comparison data can be expressed more rigorously, turning it into an optimization problem with well-defined fitness metric. Some such expressions have easily computable optimal solution.

L1 Criterion

The most natural way of expressing quality of score assignment is via accuracy - a percentage of comparisons where winning entry was assigned a higher score than the losing one. This is equivalent to minimizing the number of *upsets* (comparisons such that entry A wins against entry B, but A was assigned a lower score than B). This criterion turns out to be NP-hard [TODO: ref]. However, if we take into account the magnitude of upsets (difference between scores assigned to A and B), the problem becomes solvable in polynomial time. Minimizing the sum of magnitudes can be expressed as a linear programming problem (v_a represents the score assigned to entry A, and ϵ is added to prevent trivial solution of all scores being 0):

$$\begin{aligned} \text{minimize} \quad & \sum s_i \\ \text{subject to} \quad & v_a - v_b + s_i \geq \epsilon \\ \text{for} \quad & a > b \quad \text{in comparison set} \end{aligned}$$

This criterion can be equivalently formulated via a min-cost-max-flow problem.

1. Each node (except source and sink) represents an entry;
2. A comparison in which A wins against B is expressed as an edge from node A to node B, of capacity 1 and cost 1;
3. There's an edge from source to each node, with cost 0 and capacity equal to it's number of wins;
4. There's an edge from each node to sink, with cost 0 and capacity equal to it's number of losses.

Then, node potentials can be interpreted entry scores.

[TODO: results]

The solution to min-cost-max-flow problem also gives us an additional useful metric: percentage of "meaningful" comparisons in the comparison set. Any cycle in the network has positive cost (since all edges that don't include source and sink have positive cost), so optimal flow won't use them. Using that information, we can compute the ratio of used comparison edges (equal to total cost) to all comparisons. This quantity can be interpreted as decisiveness of comparison set - since comparison cycles can't be scored correctly, the more common they are, the less correct is it to represent LLM quality via a single, prompt-independent score.

L2 Criterion

Minimizing squares of upset magnitudes turns out to be even simpler - it is a variation of "least squares" problem. As such, this approach was chosen.

[TODO: results]

Turns out that this criterion results in a significantly better accuracy than linear one. As such, it's accuracy will be used as baseline for other models.

5.5. LLM Embedding

LLM ecosystem is constantly evolving, with new models being introduced often. While dataset selection was made with that in mind, and chosen dataset contains comparisons of LLMs that are quite new at the time of writing, constraining scoring models to only work with LLMs featured in a comparison dataset doesn't seem like a practical decision. Such datasets rely on human evaluation, and aren't easy to prepare.

An extensible alternative to scoring only LLMs whose comparison data is available, is to compute an embedding vector representing each model, and use that instead of LLM ID as scoring input data. Such embeddings can be computed without relying on comparison data, making the model easily extensible. In particular, embeddings can be computed from a set of prompts and LLM responses to them. This information is much easier to gather than human evaluations, and can be collected automatically.

An argument can be made that prompts and responses represent all information required for scoring. This way of specifying the problem transforms it into a set embedding problem: given a set of samples, define an embedding function that satisfies some requirements, while being invariant to sample ordering. Many solutions to this problem have been developed and

discussed. One such solution is described by Zaheer et. al. in *DeepSets* paper [2]. It shows that such set embedding function can be defined as a function of a mean of sample embeddings (equation 5.1). P_{out} represents final output transformation, while E_s is a sample embedding function.

$$E(S) = P_{out}(\frac{1}{|S|}\sum_{x \in S} E_s(x)) \tag{5.1}$$

This specification of the problem has an important, additional benefit. Comparisons dataset which was selected contains prompts and responses from 50 distinct LLMs. Original formulation of the problem, which focuses on assigning a single embedding vector to each LLM, only ever produces 50 different embedding values, which severely limits the dimensionality of embedding vectors before the problem becomes underspecified. By viewing LLM characteristics as only defined by responses they produce to given prompts, we can instead focus on computing embedding vectors for each prompt and response pair, raising the number of computed embedding values to 140k.

5.5.1. Metrics

Embeddings will be used by scoring models to distinguish between LLMs, and subsequently be able to determine certain characteristics of their response patterns. This means that the most important quality of LLM embeddings is distinguishability - given a subset of prompt and response pairs form a single LLM, we should be able to tell which LLM it was produced by. Rigorously, we can define an accuracy metric as follows:

TODO: equation

TODO: Other metrics?

5.5.2. Features

As described in section 5.2, text embedding vectors are used as input features. As LLM embeddings are computed from both prompts and responses, text embedding vectors of both are included in input data. Additionally, certain handcrafted response features are also included. As prompts are of secondary importance scalar prompt features described in section 5.2 were not used.

[TODO: list features]

5.5.3. DNN-based Embedding

The simplest implementation of embedding model is simply a multi-layer perceptron. *LeakyReLU* activation function was used, and dropout layers were introduced to prevent overfitting. Hidden layer sizes (and their number) are configurable.

Model was trained using triplet margin loss, given by formula [TODO: ref]. Training sample features three prompt and response pairs: anchor, positive, and negative. Positive pair is produced by the same LLM as anchor, while negative pair is produced by a different LLM. Loss criterion encourages the model to produce embeddings such that the distance between anchor and positive embeddings is smaller than between anchor and negative (plus a margin).

Additionally, KL loss was introduced to encourage a sample embeddings to follow a normal distribution.

[TODO: results]

5.5.4. Tuning-based Embedding

Text embedding vectors used by DNN embedding model are generated using a pre-trained, frozen transformer model. It can be improved by unfreezing model weights, and tuning them during training.

Tuning-based embedding model does just that. For efficiency, only last two layers of the model were unfrozen. To ensure that embedding dimension is configurable and does not depend on the choice of transformer model, projection layer was added. The same loss criterion was used as in previous approach - triplet margin and KL loss.

[TODO: It might be a good idea to improve impl there - scoring transformer does things better]

[TODO: results]

5.5.5. Attention Embedding

Set embedding problem is quite common in machine learning, and as such, a number of solutions were proposed over the years. The general idea presented in the preamble of section 5.5, defining LLM embedding as a mean of embeddings of their outputs, is loosely inspired by Zaheer et. al. *DeepSets* paper [2]. While well received and widely cited, alternatives and extending works exist. One such work is Lee et. al. *Set Transformer* paper [1]. It challenges an implicit assumption that importance of all set elements is equal. The paper proposes a relatively simple alternative to taking a mean of sample representations - attention mechanism. The rationale behind that change is that certain samples provide a much clearer information about the set

than others - capturing behaviors or characteristics that highlight differences between a given set and the rest.

Attention embedding model implements that idea. The model consists of two modules: pair encoder (illustrated by figure 5.1) and set aggregator (shown on figure 5.2). Pair encoder takes prompt and response embeddings together with scalar features as inputs, projects each into a vector of configurable dimensionality, enriches these projections with interaction features (difference between prompt and response embeddings, and their element-wise product), then runs these features through a multi-layer perceptron to produce an embedding vector.

The model can be described by equation 5.2. E_s is the sample embedding function, P_{out} - final projection module, K - key projection (linear), V - value projection (linear), and Q - query vectors (trainable parameters).

$$E(S) = P_{out}(Attention(E_s(S))) \tag{5.2}$$

$$Attention(X) = softmax(\frac{QK^T(X)}{\sqrt{dim(Q)}})V(X) \tag{5.3}$$

Set aggregator module uses sample embeddings produced by pair encoder, outputting a single LLM embedding vector. In previous embedding models, this step was done by a simple mean computation, instead. Set aggregator uses attention mechanism. Pair embeddings are used to produce key (K) and value (V) vectors through learnable linear projections. Then, attention masks are computed by a configurable number of attention heads. This mask is then used to produce a single LLM embedding from value vectors produced from pair embeddings.

The model is trained using a supervised contrastive loss. In each batch, multiple embeddings are computed for each model, using different subsets of samples. Loss criterion encourages representations of the same model to be closer to each other than to all other representations. This directly represents the distinguishability requirement - embeddings produced by different subsets of samples from the same LLM should be different than embeddings of other LLMs.

[TODO: formulas]

Results

TODO

5.5.6. Prompt-invariant Embedding-based Models

While distinguishability metric should closely correlate with LLM embedding usefulness, it can be validated by implementing a prompt-invariant scoring model which uses LLM embeddings

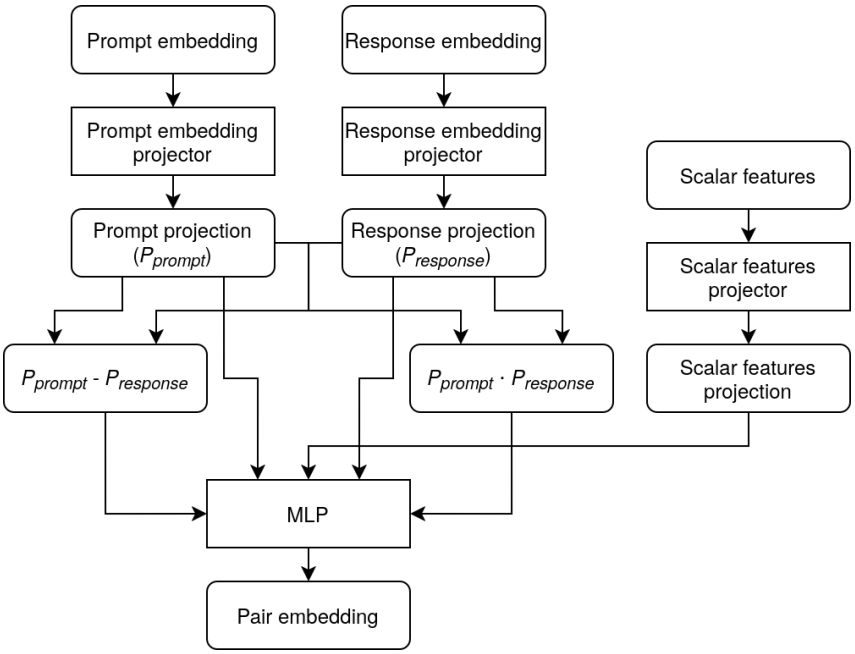

Figure 5.1: Attention embedding model - pair encoder module

instead of IDs, contrary to models described in section 5.4. If LLM embeddings are good enough, any iterative model should easily be able to reach similar accuracy to optimal one achieved by "least squares" model.

To validate that hypothesis, gradient boosting model was implemented. The model was trained using margin ranking loss.

Results

TODO

5.6. Simple Models

5.6.1. Gradient Boosting Model

[TODO: What is gradient boosting?]

Gradient boosting models are a common first choice when developing a machine learning solution for regression and classification problems. They are usually quick to learn various simple and intermediate patterns from the data. [TODO: verify] Additionally, their structure makes them relatively interpretable (especially in comparison to various neural network-based models) - as gradient boosting model is simply an ensemble of decision trees, nodes of these trees can be analyzed to find certain patterns about input to output relationships.

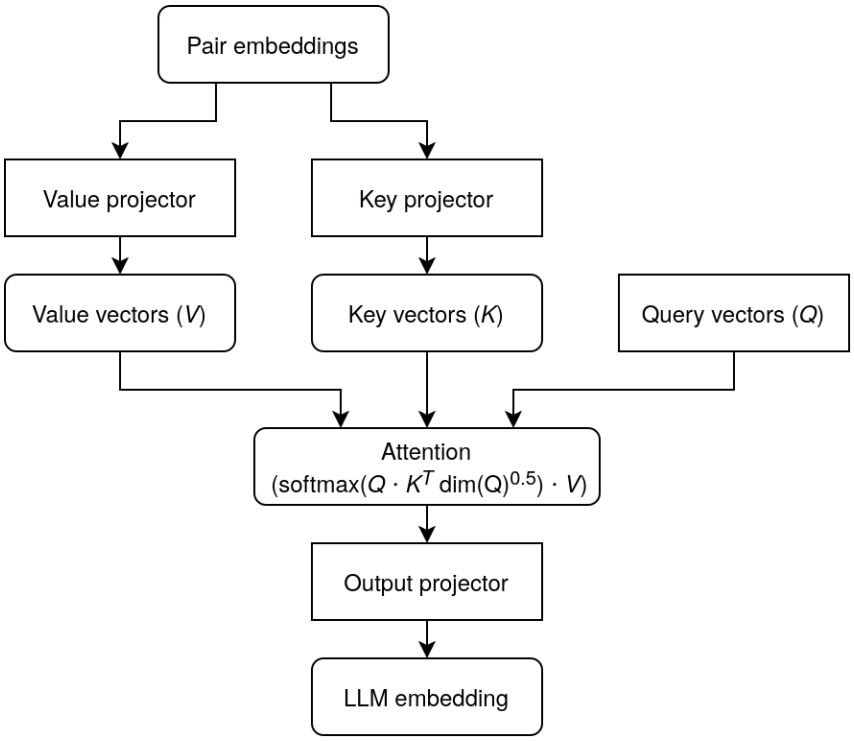

Figure 5.2: Attention embedding model - set aggregator module

Instead of implementing a separate, prompt-aware gradient boosting model, implementation described in section 5.5.6 was extended, making input data used for scoring configurable. Based on configuration, in addition to LLM embeddings model can rely on prompt embedding vectors, as well as scalar prompt features.

To match the interface of other models described further in this section, whose training process is based on *epochs*, the implementation of gradient boosting algorithm does not rely on a explicitly set number of boosting iterations - contrary to the interface offered by, for example, `XGBoost` package. This dynamic approach makes it possible to perform validations or checkpointing between epochs, and most importantly allows us to implement a mechanism which reverts model state to the one that achieved the best validation accuracy. This means that longer training process can never have a negative effect on model quality - if the model starts to overfit to training data, or hits a plateau, it will be reverted to best recorded state.

Due to it's convenience, this state reverting mechanism was implemented in all iterative models implemented during this project.

Results

TODO

5.6.2. DNN Model

The first deep learning model that was implemented uses a simple multi-layer neural network. It takes LLM embeddings, prompt embeddings, and prompt scalar features as input, and outputs a score representing how well given LLM would handle provided prompt.

Three distinct input vectors don't have to share any characteristics, like dimensionality, scale, or importance. For each, a separate projection layer was introduced. These layers transform input vectors into representations of set dimensionality, which subsequently are concatenated and used as input to the following layers of the network.

Number and sizes of hidden layers are configurable. *LeakyReLU* was chosen as the activation function, with negative slope set to 0.1. Dropout layers were introduced between each pair of hidden layers to prevent overfitting. Additionally, *LayerNorm* layers were introduced for regularization, and gradient clipping was introduced to prevent gradient explosions during training.

The score assignment goal that the network tries to achieve does not inherently require any specific scale of outputs. As scores are only ever used for comparison, transforming them through any strictly increasing function does not change the quality of the result - for each compared pair of models, bigger score will still be bigger after the transformation. This ambiguity is undesired and impractical from human perspective. Introducing a secondary training objective could be a solution to this problem, however it is unnecessary in this case. Instead of introducing an additional loss term, whose influence may negatively impact the primary objective, score post-processing was introduced. After training, mean and standard deviation of scores assigned to training data is computed, such that subsequent inference results can be normalized to mean 0 and standard deviation equal to 1. Additionally, scaled scores are transformed using a scaled hyperbolic tangent function onto a $(-1, 1)$ range.

AdamW

AdamW optimizer was used during training process. Described by Loshchilov et. al. in *Decoupled Weight Decay Regularization* paper [4], it improves the previous *Adam* algorithm (Kingma et. al., *Adam: A Method for Stochastic Optimization* [5]) by decoupling weight decay from gradient update step. In *Adam*, L2 regularization is essentially implemented by altering the effective loss criterion which is being minimized. *AdamW* implements weight decay during weight update step instead, as described by equation 5.4.

$$\theta_{t+1} = \theta_t - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} - \alpha \lambda \theta_t \tag{5.4}$$

Residual Connections

As depth of a dense neural network increases, a problem of vanishing gradient arises. [TODO: reference]. Gradient of early layer weights is a product of activation function gradients and weight values of later layers. Due to regularization and similar mechanisms, these values tend to be smaller than 1 on average, causing gradual decrease in gradient magnitude. To mitigate this issue, residual connections were introduced. Initially described by He et. al. in *Deep Residual Learning for Image Recognition* paper [3], residual connections introduce a "skip-path" for each layer. Mathematically, this means that layer input is added to layer output. The modification ensures that early layers have a direct path to network output. As vector addition can only be performed if vectors have the same dimensionality, the implementation of skip connections that relies on adding layer input to output can only be performed for layers that have the same input and output dimension. The mechanism can be easily extended to layers where these dimensions are not equal, though, by introducing a linear projection layer with the same input and output dimensions as the "primary" layer, but without activation functions, normalization layers, and so on. This extension is also described in *Deep Residual Learning for Image Recognition* paper [3], in the context of skip connections between CNN layers with different channel counts. The idea can be justified by the fact that application of multiple linear transformations is equivalent to a single linear transformation. In other words, an arbitrary early layer of the network can influence the output through a single linear transformation, which means it receives a clear gradient signal.

Loss Functions

Two interchangeable loss functions were implemented: margin ranking loss (defined by equation 5.5) and Bradley-Terry sigmoid cross-entropy loss (defined by equation 5.6). s_a and s_b represent assigned scores, $y \in \{-1, 1\}$ - comparison result, σ is the sigmoid function, and BCE is a binary cross-entropy loss function.

$$\mathcal{L}_{\text{margin ranking}}(s_a, s_b, y) = \frac{1}{n} \sum_{i=1}^{n} \max\big(0; -y_i(s_{a,i} - s_{b,i}) + m\big) \tag{5.5}$$

$$\mathcal{L}_{\text{cross-entropy}}(s_a, s_b, y) = \frac{1}{n} \sum_{i=1}^{n} \text{BCE}\left(\sigma(s_{a,i} - s_{b,i}), \frac{y_i + 1}{2}\right) \tag{5.6}$$

Initially, margin ranking loss seemed more favorable - once scores assigned to a given training pair satisfy the comparison with a margin, the pair has no contribution to loss gradients, and therefore model is not encouraged to widen the score gap. Intuitively, this is a desired behavior. Assigning a widely different scores to compared samples is by no means better than keeping

their difference small. However, margin ranking loss turned out to under-perform significantly in practice. This observation motivated the implementation of an alternative.

Metrics

Training process of every machine learning model can be monitored by following two primary metrics: loss function values, which the model is trained to minimize, and accuracy values, which, assuming good design of loss criterion, should increase as loss values decrease. Recording these values after every training epoch for both training and validation sets provides a way to verify if the model is learning correctly, overfits, stagnates, etc.

Although undeniably useful, accuracy and loss values alone are not enough to interpret the behavior of the model and to diagnose it's flaws. A set of additional diagnostic metrics was introduced to help with that. They were chosen specifically to provide information about internal network behavior. The following metrics were defined:

- **Projection variance and norm**
  Variance and L2 norm of projected values of each input component (outputs of their projection layers) are tracked. Low variance of component projections signifies it's low impact on model output - the network essentially learned to ignore any information that could be contained in that input component. Additionally, relative magnitude of projection norms can provide information about relative impact of different input components on network output.

- **Score variance decomposition**
  Variance of assigned scores is decomposed into LLM-specific and prompt-specific components. Decomposition can be described by equation 5.7 - s represents assigned scores, $\overline{s}_m$ - mean scores of LLMs, and Δs - deviations of assigned scores from the average LLM score. If nearly all variance in returned scores can be attributed to LLM differences (or, more specifically, to LLM embeddings), we can conclude that prompt information is essentially ignored.

$$Var(s) = Var(\overline{s}_m) + Var(\Delta s) \tag{5.7}$$

- **Input gradient attribution**
  During training, loss criterion gradients are computed with respect to inputs. Dot product of those gradients and input values can be used to measure importance of each input value - described by equation 5.8. This metric is especially useful, because it can be

computed not only for input components, but also for individual coordinates in a given input component vector. While per-coordinate importances don't make much sense in case of LLM and prompt embeddings, they do provide a lot of information in case of scalar features - features that are not useful for scoring can be easily identified by their low importance.

$$Imp(x) = \nabla_x L \cdot x \tag{5.8}$$

Post-training Sensitivity Analysis

In addition to metrics described in section 5.6.2, which are computed and tracked per epoch, additional post-training sensitivity metrics were implemented. While the primary goal of previously defined metrics is training process monitoring and diagnosis of certain undesired patterns, post-training metrics described in this section are implemented to provide an interpretation of the scoring behavior of a trained model. While it would be possible to compute these metrics after every epoch as well, to provide an insight into how scoring behavior is changing during training, doing so proved impractical due to relatively high computational cost. Measuring each metric requires at least one additional pass through the training set.

The following sensitivity metrics were implemented:

- **Prompt sensitivity**

  Prompts are randomly permuted across all samples, and model accuracy is measured. Drop in accuracy can be interpreted as model's sensitivity to prompt information - if the model learned to rely on LLM embeddings, the accuracy won't change significantly, and if the model does consider prompt inputs, the drop will be significant.

- **Prompt embedding sensitivity**

  Model accuracy is measured after setting all prompt embeddings to their mean across the sample set. Drop in accuracy can be interpreted as model's sensitivity to prompt embeddings, specifically.

- **Scalar features sensitivity**

  Model accuracy is measured after setting all scalar feature inputs to their mean across the sample set. Drop in accuracy can be interpreted as model's sensitivity to scalar features.

- **Per-feature sensitivities**

  Model accuracy is measured after randomly permuting values of a single scalar feature

across the sample set. Drop in accuracy can be interpreted as model's sensitivity to that feature.

Each metric is computed separately on training and validation sets.

Results

TODO

5.7. Complex Models

5.7.1. Transformer Model

Sentence encoding transformers are a useful tool that transform text into a fixed-dimensional vector encoding it's meaning, hopefully in a way that's useful for subsequent machine learning tasks. The main input data of the scoring task is the prompt for which an LLM needs to be selected, and as such pretrained sentence transformers seem like a good candidate for the main, trainable part of the scoring model.

The core of the transformer model is a pretrained sentence transformer model, which is being fine-tuned during the training process. The output of the transformer model is passed to a scoring head, which consists of a small (configurable) number of dense layers, using *LeakyReLU* activation functions (with negative slope equal to 0.1) and dropout layers. Scoring head module also takes LLM embeddings and scalar prompt features as inputs (since, naturally, sentence transformer module only takes text as input). These input vectors are initially passed through projection layers, similarly to what is described in 5.6.2.

The core transformer model is loaded from *HuggingFace* using `sentence-transformers` library. [TODO: something more]. Wide variety of available models make it possible to test different options, with varying parameter counts, output dimensionalities, and architectures.

The same loss functions were implemented as in previously described models (in section 5.6.2). Furthermore, the same arguments prompted the decision to leave score values unconstrained during training, adding normalization and hyperbolic tangent transformation as a post-processing step, instead. Transformer model also uses the same state reverting mechanism, preventing loss of quality due to overfitting. Additionally, since training process of this model is significantly more time-consuming, checkpointing was implemented, saving model state every few epochs, which proved useful in case of crashes during training (mostly due to out-of-memory errors).

Fine-tuning Methods

A number of fine-tuning methods were implemented. Initially, the easiest approach involved simply unfreezing n final layers of the model. This turned out to produce terrible results, while also not reducing the number of trainable parameters enough. [TODO: something more]. *LoRA* and *QLoRA* methods were implemented as alternatives. The main idea behind *LoRA* (or *Low Rank Adaptation*) is the introduction of a smaller number of new, trainable parameters, while keeping original model weights frozen. Effective, altered model weights are computed according to equation 5.9, where r is a configurable rank of the transformation. [TODO: ref + some justification?]

$$W' = W + AB \tag{5.9}$$

$$W \in R^{m \times n} \tag{5.10}$$

$$A \in R^{m \times r} \tag{5.11}$$

$$B \in R^{r \times n} \tag{5.12}$$

QLoRA extends *LoRA* method by also introducing model weight quantization. By default, transformer models usually use 16-bit floating point numbers as weights. QLoRA performs 8-bit quantization, followed by a 4-bit quantization using *NormalFloat4* technique [TODO: ref]. *NormalFloat4* encoding is a theoretically optimal way of encoding normally distributed values. While there is no theoretical reason to expect network weights to be normally distributed (although, perhaps, an argument could be made that this expectation is justified by Central Limit Theorem [TODO: ref], treating individual weight updates as random variables), this quantization method does seem effective in practice [TODO: ref]. Only the original model weights are quantized - new, trainable parameters from LoRA matrices retain their full, 16-bit representation. In addition to quantization, QLoRA also implements optimizer state pagination. To avoid out-of-memory issues during training, optimizer parameters (momentum, cache, etc.) that are not in active use are transferred to CPU memory. This results in a time overhead caused by numerous data transfers between GPU and CPU memory, but allows training larger models on a given architecture. The comparison of *QLoRA* and other described fine-tuning is shown on figure 5.3.

Interaction Features

The model is designed to learn most information required for score prediction through fine-tuning. This decision dictates the small size of scoring head module - it's only purpose is to

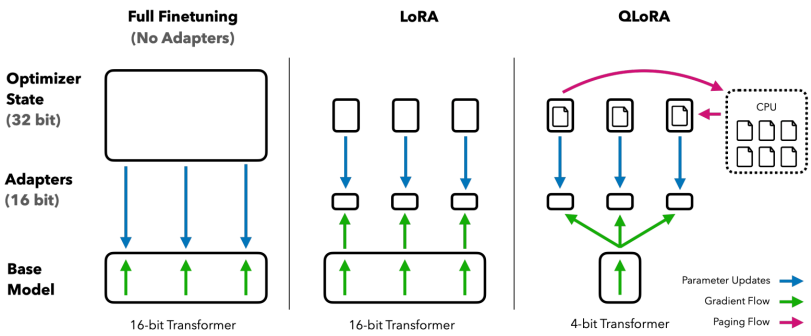

Figure 5.3: Fine-tuning methods comparison

combine LLM embedding vectors (and scalar prompt features, if they turn out to be useful) with prompt embeddings. However, this introduces certain risks. The "path length", or, more formally, number of operations, between prompt text and output is significantly bigger than between other inputs and output. This may cause the model to initially take the path of least resistance, focusing only on LLM embeddings (essentially learning optimal prompt-invariant score for each vector), and discarding prompt embeddings entirely, essentially blocking gradient flow to the transformer module.

To address this issue, simple concatenation of projections of scoring head input vectors was replaced by another combining operation. The goal is to discourage the model from learning scores that depend only on LLM embeddings, and instead encourage it to produce scores based on certain interactions between prompt representation and LLM embedding. To encode that interaction, scalar product was used. Scoring head module takes as input the scalar product of LLM embedding projection, and concatenated prompt embedding and scalar prompt features projections. As scalar product requires arguments to be of the same dimensionality, projection layer sizes were adjusted. Size of scalar features projection is configurable, with prompt embedding projection being twice as big, and LLM embedding projection - three times as big.

Of course, this approach does not guarantee that the model cannot ignore prompts. [TODO: handle zeroing prompt text input while keeping prompt features input]. To achieve that, while retaining LLM embedding information, transformer module would need to learn to output a constant, non-zero vector for every prompt. Importantly, though, this state doesn't block the gradient flow to transformer module - even if the model initially does learn to ignore prompt information, it should, theoretically, be able to continue learning prompt-specific relationships after reaching optimal prompt-invariant scoring accuracy.

Loss Functions

The model uses the same two ranking loss function variants as dense network model (in section 5.6.2) - margin ranking loss, and Bradley-Terry cross-entropy loss.

Metrics

The same diagnostic metrics were introduced as in dense network model. As fine-tuned transformer module takes text as input, gradient attribution is computed for prompt embedding vectors that it outputs.

Additionally, a new metric was introduced: module gradient norms. While dense network model consists of a single main component (dense network) and input projection layers, there are two high-parameter, trainable modules in transformer model - the actual sentence transformer being fine-tuned, and scoring head (dense network). By computing and recording norms of gradients of parameters of each module, it's possible to validate that both keep learning. For example, if the model learns to rely only on LLM embeddings, ignoring prompt information, transformer module gradient norms will significantly decrease relatively to scoring head gradient norms.

Muon optimizer

Instead of *AdamW* optimizer, which was used for dense network model training, transformer model uses *Muon* optimizer. Introduced by Jordan et. al. in an article titled *Muon: An Optimizer for Hidden Layers in Neural Networks* [6], and later evaluated in context of LLM training by Liu et. al. in *Muon is Scalable for LLM Training* paper [7], *Muon* changes how momentum influences weight updates. While *AdamW* only uses element-wise operations to update weights, *Muon* exploits the matrix structure of most network parameters.

Algorithm 1 describes weight update process. After gradient is computed and momentum is updated, orthogonalization is performed on the result by computing a polar factor $X = UV^T$, from singular value decomposition $M = (UV^T)(V\Sigma V^T)$. The result X is a semi-orthogonal matrix, which captures the direction of required weight updates while normalizing the magnitude. This step effectively replaces the per-element second moment estimate ($\hat{v}_t$) normalization in *AdamW* algorithm. Additionally, resulting matrix X has all singular values equal to 1, so weight update O_t has a bounded spectral norm regardless of how large the gradient was. This means that orthogonalization acts as implicit regularization.

Before orthogonalization is performed, Nesterov lookahead momentum correction is applied (Y. Nesterov, *Introductory Lectures on Convex Optimization: A Basic Course* [8]). As magnitude

information is discarded, direction accuracy becomes more important, making the correction more important compared to previous optimizing algorithms, like *Adam*.

Orthogonalization is performed using Newton-Schulz iterative method. It is an approximate, inversion-free alternative to computationally expensive exact singular value decomposition methods.

Algorithm 1 Muon optimizer (for 2D hidden-layer parameters)

Require: learning rate η, momentum μ, weight decay λ, NS steps $K = 5$, NS coefficients (a, b, c)

Initialize momentum buffer $M_0 \leftarrow 0$
for $t = 1, 2, \ldots$ **do**
  $G_t \leftarrow \nabla_\theta \mathcal{L}(\theta_{t-1})$ {compute gradient}
  $M_t \leftarrow \mu\, M_{t-1} + G_t$ {momentum accumulation}
  $\hat{M}_t \leftarrow M_t + \mu\,(M_t - M_{t-1})$ {Nesterov lookahead (optional)}
  $X \leftarrow \hat{M}_t$ {begin Newton–Schulz orthogonalization}
  $X \leftarrow X \,/\, \|X\|_F$ {normalize for NS convergence}
  for $k = 1, \ldots, K$ **do**
    $X \leftarrow aX + b(XX^\top)X + c(XX^\top)^2 X$
  end for
  $O_t \leftarrow X$ {$O_t \approx UV^\top$ from polar decomposition of $\hat{M}_t$}
  $\theta_t \leftarrow \theta_{t-1} - \eta\, O_t - \eta\lambda\, \theta_{t-1}$ {update with decoupled weight decay}
end for

Results

TODO

5.7.2. Response Predictive Model

Models described in previous sections have one common characteristic: they do not use LLM responses during training, despite them being available in the dataset. The reason of that is quite obvious - LLM responses are not accessible at inference time. The whole point of assigning scores to LLMs is to be able to decide which will produce the best response, without actually requesting that response. Intuitively, if LLM responses were available, the task should be significantly easier. Assigning scores based only on the prompt and information about chosen LLM internally requires deducing certain response characteristics, based on which scoring can subsequently be performed.

The main idea of response predictive model is to utilize response data as much as possible, without requiring it for inference. This is achieved by dividing the scoring task into two parts, described in previous paragraph. First module, the predictor, is responsible for predicting response representation, given information about the prompt and chosen LLM. Second module, the scorer, is responsible for assigning a score, based only on that representation (without additional information about the LLM). End-to-end process retains the same input/output interface as other models, taking prompt and LLM data as input, and returning a score. However, explicit separation of evaluation steps opens new possibilities for training.

Prompt and LLM representations used by response predictive model are the same as in other models. Prompts are represented by text embedding vectors and scalar features, while LLMs are encoded as embedding vectors. As this model also uses LLM responses, they need to be represented somehow. The simplest possibility would be to use a similar representation to what is described in section 5.5.2. This representation might be useful for scoring, however there is no guarantee that it would be possible to predict it from just prompt and LLM data. In general, predicting LLM responses is an extremely hard task - predicting all characteristics of the response would be essentially equivalent of generating the response, requiring parameter counts and training data volumes orders of magnitude larger than what's achievable in this project. So, response representation should be carefully tuned in such a way that it is still useful for the scoring module, while also being predictable by the predictor module.

Finding such a representation can be incorporated into the training process. A third module is introduced: response encoder. Given response representation (in format described in section 5.5.2), it learns to produce response embedding vectors.

Importantly, predictor and response encoder operate on different inputs. Response encoder has access only to real LLM response representations, while predictor has no information about them. As responses are not available during inference, response encoder is not used in that case. It's purpose is to help in discovering response representations that can be useful for scoring. Having access to actual responses, finding such embeddings should be much easier.

Modular structure of response predictive model makes it more interpretable than most other models, which usually function as black-box predictors. Each of the three modules has it's separate role, making it easy to measure the quality with which it performs it's task. In particular, we can decouple prediction errors from representation and scorer errors, by comparing scoring accuracy when response embeddings are predicted, with scoring accuracy using real response embeddings. Similarly, predictor quality can be tracked by measuring common regression metrics, like mean squared error or cosine similarity, and by comparing distributions of real and predicted

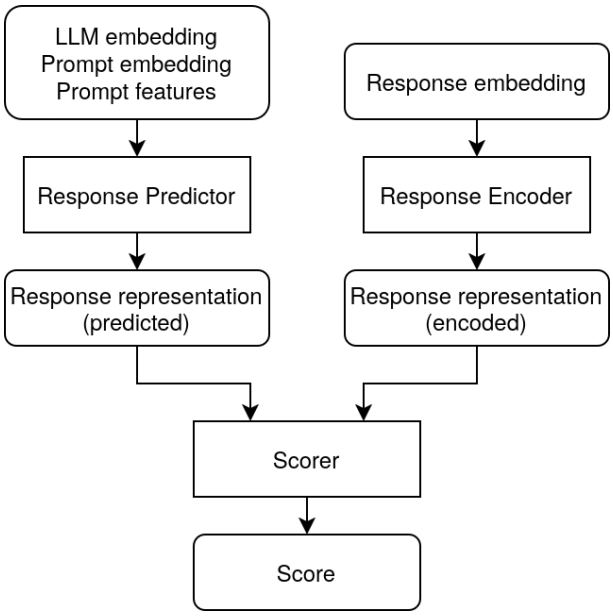

Figure 5.4: Response predictive model architecture

embeddings - ideally, if predictor and encoder modules find some representation equilibrium, distribution should be similar.

Loss Function

Due to relative complexity of training process of response predictive model, it requires a more complex loss function design than other models. Using a simpler design (i.e. just ranking loss) ultimately leads to a "collapse" to a simpler, locally optimal state. For example, if no effort is made to ensure predicted and encoded representations are similar, scoring module can just learn to treat both types of representations differently.

Four loss components are combined, according to equation 5.13. Weights k_{pred}, k_{repr} and k_{KL} are configurable.

$$\mathcal{L}_{\text{response predictive}} = \mathcal{L}_{\text{rank}} + k_{\text{pred}}\mathcal{L}_{\text{pred}} + w(n_{\text{epoch}})k_{\text{repr}}\mathcal{L}_{\text{repr}} + \tag{5.13}$$

$$+k_{\text{anchor}}\mathcal{L}_{\text{anchor}} + k_{\text{consistency}}\mathcal{L}_{\text{consistency}} + k_{\text{dist}}\mathcal{L}_{\text{dist}} \tag{5.14}$$

First criterion is ranking loss. Two interchangeable ranking loss options were defined, the same as in previously described models: margin ranking loss (equation5.5) and Bradley-Terry sigmoid cross-entropy (equation 5.6). Loss criterion is computed not only using predictor module flow (where response representations are predicted), but also through encoder flow, using real responses as input.

Second criterion encodes prediction accuracy. It encourages predictor module to produce outputs similar to those produced by response encoder. Encoder outputs are treated as constants in this case - gradients are computed only with respect to predictor weights. Third criterion is the same as second, but treating predictor outputs as constants instead. It instructs response encoder module to produce representations that predictor module is able to predict. Three interchangeable variants were defined for this loss component: MSE (equation 5.15), cosine similarity (equation 5.16) and Huber L1 criterion (equation 5.17). $\hat{r}$ represents predicted response representations, and r - encoded response representations. The sum in Huber criterion iterates over response vector components.

$$\mathcal{L}_{\text{MSE}} = \frac{1}{n}\sum_{i=1}^{n}\left|\hat{r}_i - r_i\right|^2 \tag{5.15}$$

$$\mathcal{L}_{\text{cosine}} = \frac{1}{n}\sum_{i=1}^{n}\left(1 - \frac{\hat{r}_i \cdot r_i}{|\hat{r}_i||r_i|}\right) \tag{5.16}$$

$$\mathcal{L}_{\text{huber}} = \frac{1}{d_r}\sum_{j=1}^{d_r}\begin{cases}\frac{1}{2}(\hat{r}_{i,j} - r_{i,j})^2 & |\hat{r}_{i,j} - r_{i,j}| \leq 1 \\ |\hat{r}_{i,j} - r_{i,j}| - \frac{1}{2} & \text{otherwise}\end{cases} \tag{5.17}$$

Predictor loss component is additionally weighted by function $w(n) = \max(1, \frac{n}{n_w})$, where n is the current epoch number. At the beginning of training, response representations produced by predictor module are highly random. There's no point in training response encoder to produce outputs that are similar to such random noise. $w(n)$ gradually ramps up the importance of that loss criterion as training progresses and predictor starts learning.

Fourth criterion is given by equation 5.18. It is a KL loss term, encouraging a normalized distribution of response embeddings. In particular, this prevents initial representation collapse to a single value - it is a trivially optimal solution to the second and third loss criterion. Importances of loss criteria are controlled by configurable weights. [TODO: cite something + explain]

$$\mathcal{L}_{\text{anchor}} = \frac{1}{d_r}\sum_{j=1}^{d_r}\frac{1}{2}\left(\sigma_j^2 + \mu_j^2 - 1 - \ln\sigma_j^2\right) \tag{5.18}$$

Fifth criterion is given by equation 5.19. It encourages both encoder and predictor to produce representations that would be scored similarly by the scorer model.

$$\mathcal{L}_{\text{consistency}} = \frac{1}{n}\sum_{i=0}^{n}(Score(\hat{r}_i) - Score(r_i))^2 \tag{5.19}$$

Last criterion is given by equation 5.20. It is a sum of Jeffrey's divergences between encoded and predicted values of each response representation component. [TODO: cite something + explain]

$$\mathrm{KL}\big(\mathcal{N}(\mu_1, \sigma_1^2)|\mathcal{N}(\mu_2, \sigma_2^2)\big) = \frac{1}{2}\left(\frac{\sigma_1^2}{\sigma_2^2} + \frac{(\mu_1 - \mu_2)^2}{\sigma_2^2} - 1 + \ln\frac{\sigma_2^2}{\sigma_1^2}\right) \tag{5.20}$$

$$\mathcal{L}_{\text{dist}} = \frac{1}{d_r}\sum_{i=1}^{d_r}\frac{\mathrm{KL}(\hat{p}_j|p_j) + \mathrm{KL}(p_j|\hat{p}_j)}{2} \tag{5.21}$$

Results

TODO

5.7.3. Multistage Models

Models described in this section are focusing on assigning a score based on prompt and LLM information. In some cases, it's possible to at least partially interpret their decisions ([TODO: ref]), but in general this formulation of the task does not imply any interpretability. To help with that, model chaining mechanism was introduced. Taking advantage of the fact that scores learned during training are not constrained in any way, given a trained scoring model as "base", another can be fitted on it's errors, attempting to predict the difference between it's predecessor and the true value. Final scores are defined as the sum of both predictions.

We can describe model chaining via equation 5.22. The main interpretability benefit comes from a possibility of constraining input values passed to each model - I_k for a model given by $Score_k$. By gradually extending the set of input components, impact of introduction of each component is clearly visible as an increase in joint model performance quality.

$$Score(I) = \sum_k Score_k(I_k), \quad \forall_k I_k \subset I \tag{5.22}$$

In LLM scoring task, multi-stage models can be especially useful when a prompt-invariant model is used as base. As described in section 5.4, optimal score assignment based only on chosen LLM is rather simple. By using such model and training a more complex model to predict prompt-specific differences, we clearly separate accuracy and score contributions that use prompt information from base LLM-only "bias".

An additional advantage of multi-stage approach is faster training. Complex models may take multiple epochs before reaching the optimal prompt-invariant accuracy. If prompt-invariant model is used as base, however, optimal prompt-invariant accuracy is the starting point.

TODO: results

6. Response Length Prediction

TODO: Should it be a separate chapter?

While predicting how well different LLM models can handle a given prompt is the most important part of the LLM selection process, there is a second unknown quantity that influences the choice: response length. While pricing rules of LLM requests are usually clear, cost dependence on response token count makes it impossible to accurately calculate the cost of a given request beforehand. Generally, this isn't a significant problem - output tokens are relatively cheap, especially in the context of requests including images and files in the input. TODO: Continue

6.1. Input and Output Data

TODO

6.2. Models

6.2.1. Gradient Boosting Model

TODO

6.2.2. DNN Model

TODO

7. Analysis of the Solution

TODO: Tests of all parts of the system, etc.

8. Conclusion

TODO

Bibliography

[1] J. Lee, Y. Lee, J. Kim, A. R. Kosiorek, S. Choi, Y. W. Teh, *Set Transformer: A Framework for Attention-based Permutation-Invariant Neural Networks*, Proceedings of the 36th International Conference on Machine Learning (ICML), PMLR 97, 2019.

[2] M. Zaheer, S. Kottur, S. Ravanbakhsh, B. Poczos, R. R. Salakhutdinov, A. J. Smola, *Deep Sets*, Advances in Neural Information Processing Systems (NeurIPS), vol. 30, 2017.

[3] K. He, X. Zhang, S. Ren, J. Sun, *Deep Residual Learning for Image Recognition*, IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016.

[4] I. Loshchilov, F. Hutter, *Decoupled Weight Decay Regularization*, International Conference on Learning Representations, 2019.

[5] D. P. Kingma, J. Ba, *Adam: A Method for Stochastic Optimization*, International Conference on Learning Representations, 2015.

[6] K. Jordan, Y. Jin, V. Boza, J. You, F. Cesista, L. Newhouse, J. Bernstein, *Muon: An Optimizer for Hidden Layers in Neural Networks*, 2024, `https://kellerjordan.github.io/posts/muon/`.

[7] J. Liu, J. Su, X. Yao, Z. Jiang, G. Lai, Y. Du, Y. Qin, W. Xu, E. Lu, J. Yan, *Muon is Scalable for LLM Training*, arXiv preprint arXiv:2502.16982, 2025.

[8] Y. Nesterov, *Introductory Lectures on Convex Optimization: A Basic Course*, Springer, 2004.

List of symbols and abbreviations

nzw.	nadzwyczajny
*	star operator
~	tilde

If you don't need it, delete it.

List of Figures

4.1 Task execution flow 16
4.2 Utility function values 22

5.1 Attention embedding model - pair encoder module 35
5.2 Attention embedding model - set aggregator module 36
5.3 Fine-tuning methods comparison 43
5.4 Response predictive model architecture 47

If you don't need it, delete it.

Spis tabel

If you don't need it, delete it.

List of appendices

1. Appendix 1
2. Appendix 2
3. In case of no appendices, delete this part.